

Prompt Engineering Activity for ITE153

I. Overview & Importance

In the modern IT landscape, Large Language Models (LLMs) are no longer just novelty tools; they are foundational components of software development, system administration, and data analysis. However, an AI model is only as effective as the instructions it receives.

Prompt Engineering is the practice of structuring text inputs to ensure LLMs produce accurate, context-aware, and structured outputs. For IT professionals, mastering this skill is critical for automating workflows, parsing logs, generating code, and building AI-driven applications. This lab will introduce you to core prompt engineering techniques through hands-on experimentation and iterative refinement.

II. Learning Objectives

By the end of this lab exercise, you will be able to:

- **Analyze** the differences between vague and well-structured prompts.
- **Apply** foundational prompt engineering techniques, including **Role Prompting**, **Delimiters**, **Few-Shot Prompting**, and **Chain-of-Thought (CoT)**.
- **Design and refine** prompts to transform unstructured IT data into structured, actionable formats.
- **Evaluate** LLM outputs critically, identifying inconsistencies, biases, or hallucinations.

III. Prerequisites & Tools

- Access to a web-based LLM user interface (e.g., OpenAI ChatGPT, Anthropic Claude, Google Gemini, or Microsoft Copilot).
- A text editor or markdown file to document your prompts and the model's responses.

IV. Lab Activity: The IT Support Ticket Automation Pipeline

Scenario Background

You are a Junior IT Operations Analyst at *CloudSync Solutions*. The Helpdesk department is overwhelmed with raw, unstructured support tickets submitted via email. Your manager wants to automate the triage process using an AI model.

Your task is to develop a prompt that takes a messy, emotional user email and extracts three specific pieces of information:

1. **Urgency Level:** (Low, Medium, High, Critical)
2. **Affected Subsystem:** (Network, Database, Authentication, Hardware, Software)
3. **Root Cause Summary:** A technical summary of the issue in 15 words or less.

Unstructured Data Asset (Copy this text for the exercises below):

"Hey Support, I'm losing my mind here. I've been trying to log into the internal inventory portal since 8:00 AM because I need to ship out an urgent order for our biggest client. Every time I hit the login button after typing my password, it just spins for two minutes and then gives me a '504 Gateway Timeout' error. My teammate Sarah tried it too and got the same thing, so it's not just my machine. Fix this ASAP or we are going to lose this contract!!"

Task 1: The Naive Prompt (Baseline)

1. Open your chosen LLM.
2. Write a simple, one-sentence prompt asking the AI to read the support ticket above and give you the urgency, subsystem, and summary.
 - *Example naive prompt:*

"Tell me what's wrong with this ticket and how urgent it is: [Insert Ticket Text]"
3. **Deliverable:** Paste your exact prompt and the model's response into your lab report. Note how the output is formatted. Is it easy for an automated script to parse?

Task 2: Adding Structure and Role Prompting

Let's improve the output by defining the AI's role, using clear delimiters to separate instructions from data, and constraining the output format.

1. Craft a new prompt using the following framework:
 - **Role:** Act as a Senior IT Operations Triage Bot.
 - **Context/Task:** Analyze the provided IT support ticket.
 - **Constraints:** You must only choose from the predefined categories for Urgency (Low, Medium, High, Critical) and Subsystem (Network, Database, Authentication, Hardware, Software).
 - **Output Format:** Request the output strictly as a **JSON object** with three keys: urgency, subsystem, and summary.
 - **Delimiters:** Wrap the raw ticket data in triple backticks (```) or XML-style tags (e.g., <ticket>...</ticket>).
2. Run this prompt and record the result.
3. **Deliverable:** Document your refined prompt and the resulting JSON output.

Task 3: Few-Shot Prompting for Edge Cases

Sometimes models struggle to categorize subtle technical nuances. For instance, the user mentioned a "login button" (which sounds like Authentication), but received a "504 Gateway Timeout" (which is a Network/Server issue).

To ensure the model correctly identifies this as a **Network** issue rather than an Authentication issue, you need to provide examples (*Few-Shot Prompting*).

1. Modify your prompt from Task 2 to include **two examples** of properly classified tickets before feeding it the actual task.
 - *Example 1:*
A user complaining about a forgotten password → Subsystem: Authentication.
 - *Example 2:*
A user complaining about a slow page load and 502/504 errors → Subsystem: Network.
2. Run the updated prompt with the original unstructured asset.
3. **Deliverable:** Paste your few-shot prompt and evaluate whether the model's classification changed or became more accurate.

V. Deliverables & Submission Layout

Submit a single PDF document structured as follows:

Section	Required Content	Expected Detail
Student Info	Name, Student ID, Section, Date	Standard header
Task 1	Naive Prompt text + AI Response	Raw, unformatted output observation
Task 2	Structured/Role Prompt text + JSON Response	Examination of constraint adherence
Task 3	Few-Shot Prompt text + Final JSON Response	Demonstration of learning via examples
Reflection	Answers to the Critical Thinking Questions	Deep-dive analysis

VI. Reflection & Critical Thinking Questions

Answer the following questions in your submission document based on your observations during the activity:

1. **The Impact of Constraints:** How did transitioning from the naive prompt (Task 1) to the structured JSON prompt (Task 2) change the predictability and usability of the AI's response for an IT workflow?
2. **Handling Ambiguity:** In the case study, the user mentioned "logging in," but the actual error was a network timeout. Did your prompt successfully guide the AI to focus on the root cause rather than the surface-level keyword? Explain how your prompt design influenced this.
3. **Determinism and Iteration:** Did the AI give you the exact same answer every time you ran the same prompt? Why is understanding this behavior crucial when designing AI components for enterprise IT systems?

VII. Grading Rubric (100 Points Total)

- **Task 1 Execution (15 pts):** Successfully documents a baseline naive prompt and captures raw output.
- **Task 2 Prompt Design (25 pts):** Correctly implements Role Prompting, delimiters, and forces a valid, clean JSON structure.
- **Task 3 Advanced Engineering (25 pts):** Effectively uses Few-Shot prompting to handle technical nuances and edge cases.
- **Reflection & Critical Thinking (25 pts):** Displays deep understanding of prompt engineering principles, formatting constraints, and LLM behavior in an IT context.
- **Professionalism & Formatting (10 pts):** Document is organized, clean, free of typographical errors, and easy to follow.